

Similarity Search

Find what's similar, not just what matches

1 Represent (Embeddings)

Convert text, image, audio, etc. into a numeric vector (embedding) that captures meaning.

"A cute cat"

Embedding Model
(e.g., OpenAI, BERT, CLIP, etc.)

[0.12,
-0.45,
0.33,
...]

Similar meaning → similar vectors (close in vector space)

2 Normalize

Normalize embeddings (usually to unit length) so similarity comparison becomes consistent and fair.

Original vector

[0.6, 0.8]

Normalize
(to unit length)

Normalized vector

[0.6, 0.8]
(norm = 1)

3 Measure Similarity

Calculate how close two vectors are using a distance or similarity metric.

Cosine Similarity

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$$



Measures angle (common for text)

Dot Product

$$A \cdot B = \sum A_i B_i$$

Fast and works well
(if vectors are normalized)

Euclidean Distance (L2)

$$\|A - B\|_2 = \sqrt{\sum (A_i - B_i)^2}$$

Measures straight-line distance
(smaller = more similar)

4 Find Neighbors

Given a query vector, find the top k most similar vectors (neighbors) from a large collection.

Nearest neighbors (top k)

Query Vector

Two approaches:

Exact kNN

- Brute force search
- Accurate
- Slower (checks every vector)

Approximate ANN

- Faster, scalable
- Slightly less accurate
- Popular methods:
 - HNSW
 - IVFPQ

5 Compare to Keyword Search (BM25)

Keyword Search (BM25)



- Matches exact words
- Lexical (keyword) based
- Great for known terms
- Fails for meaning/synonyms

Vector Search (Similarity Search)



- Matches by meaning
- Semantic (vector) based
- Finds related content (even without exact words)
- Better for fuzzy, natural language queries

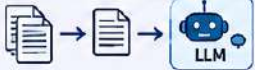
Hybrid Search + Rerank

Combine keyword (BM25) + vector search and rerank results (e.g., using a cross-encoder) for best accuracy.

6 Powers (Real-World Applications)

Similarity search enables many powerful use cases:

RAG (Retrieval Augmented Generation)



Retrieve relevant context for better AI answers

Recommendations



Find similar products, movies, or content

Deduplication / Clustering



Find and remove duplicates or group similar items

Anomaly Detection



Find unusual items (outliers)



**Same idea.
Many possibilities.**

Represent → Normalize → Measure →
Find Neighbors → Compare → Real-World Impact

No Shared Words, Close in Meaning

Different words, same intent — similarity search understands the meaning.



User Query

“reset my password”

Keyword Search ❌

Matches exact words from the query.



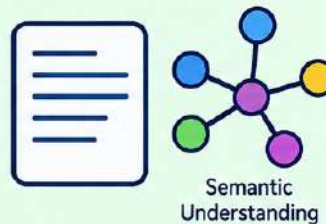
Misses

“Recover account access”

No matching words, so it doesn't find the result.

Similarity Search ✅

Understands semantic meaning using embeddings.



Finds

“Recover account access”

Even though the words are different, the meaning is close.



Different words → Same meaning → **Similarity Search finds it**

Keyword-Only Knowledge Base: The Problem

Different words, same question — but no match!

1 Keyword-only knowledge base

Docs are indexed using exact keywords (no understanding of meaning).

Reset Password Guide

Learn how to reset your password in simple steps.

Change Password

Steps to change your password.

Forgot Password

What to do if you forgot your password.

Only exact keywords are matched.

2 User paraphrases the question

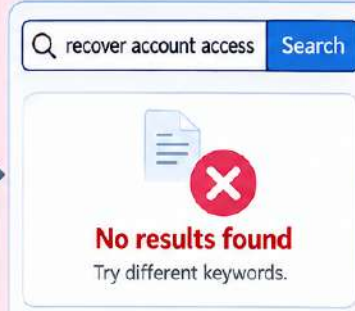
The user asks the same question in a different way.



Different words, same intent.

3 Zero keyword overlap

No exact keyword match in the knowledge base.



The system looks for exact words, and since there is no overlap, it can't find the answer.

4 User frustrated / bot says "I don't know"

The bot fails to help, leading to a poor user experience.



The user leaves disappointed and doesn't get the help they need.

5 Add synonym lists by hand

You can try adding synonyms manually (e.g., "reset" → "recover", "account" → "login").

Synonyms (manual)

reset → recover
password → login
account → user
forgot → lost
...

Endless, brittle, never complete

- Many ways users can phrase the same thing
- Hard to maintain
- Always miss new variations
- Doesn't scale

People use different words. But they mean the same thing!

How Similarity Search Works

Index once, then search for every request

Same model for docs and query! 💡

1

Index — done once / on update

Convert documents into vectors and store them in a vector index.

Docs / chunks



Your documents
(split into chunks)

Normalize



Clean and normalize
text (lowercase,
remove extra spaces,
etc.)

Embed
(same model!)



Convert each chunk
to a vector using
the same embedding
model.

e.g., [0.12, -0.45, 0.33, ...]

Search index



Store vectors +
metadata in a
vector index
(e.g., HNSW, IVFPQ).



Stores document
vectors for
fast search

2

Query — every request

Convert the user query to a vector and find the most similar documents in the index.

User query



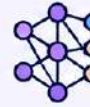
User enters
a question

Normalize



Clean and normalize
the query (same
way as documents).

Embed
(same model!)



Convert query
to a vector using
the same embedding
model.

e.g., [0.11, -0.40, 0.28, ...]

Top-k nearest
neighbors



Find the top-k
most similar
documents from
the vector index.

Results

- 1 Reset your password
- 2 Account recovery options
- 3 Troubleshooting login issues



Key Points

- Indexing is done once (or when docs update).
- Use the same embedding model for both documents and queries.
- At query time, find the top-k nearest neighbors from the vector index.
- Return the most relevant chunks to the user (often with a reranker).



Common Vector Indexes

- HNSW (fast, high recall)
- IVFPQ (scalable, memory efficient)
- Others: FAISS, ScaNN, etc.



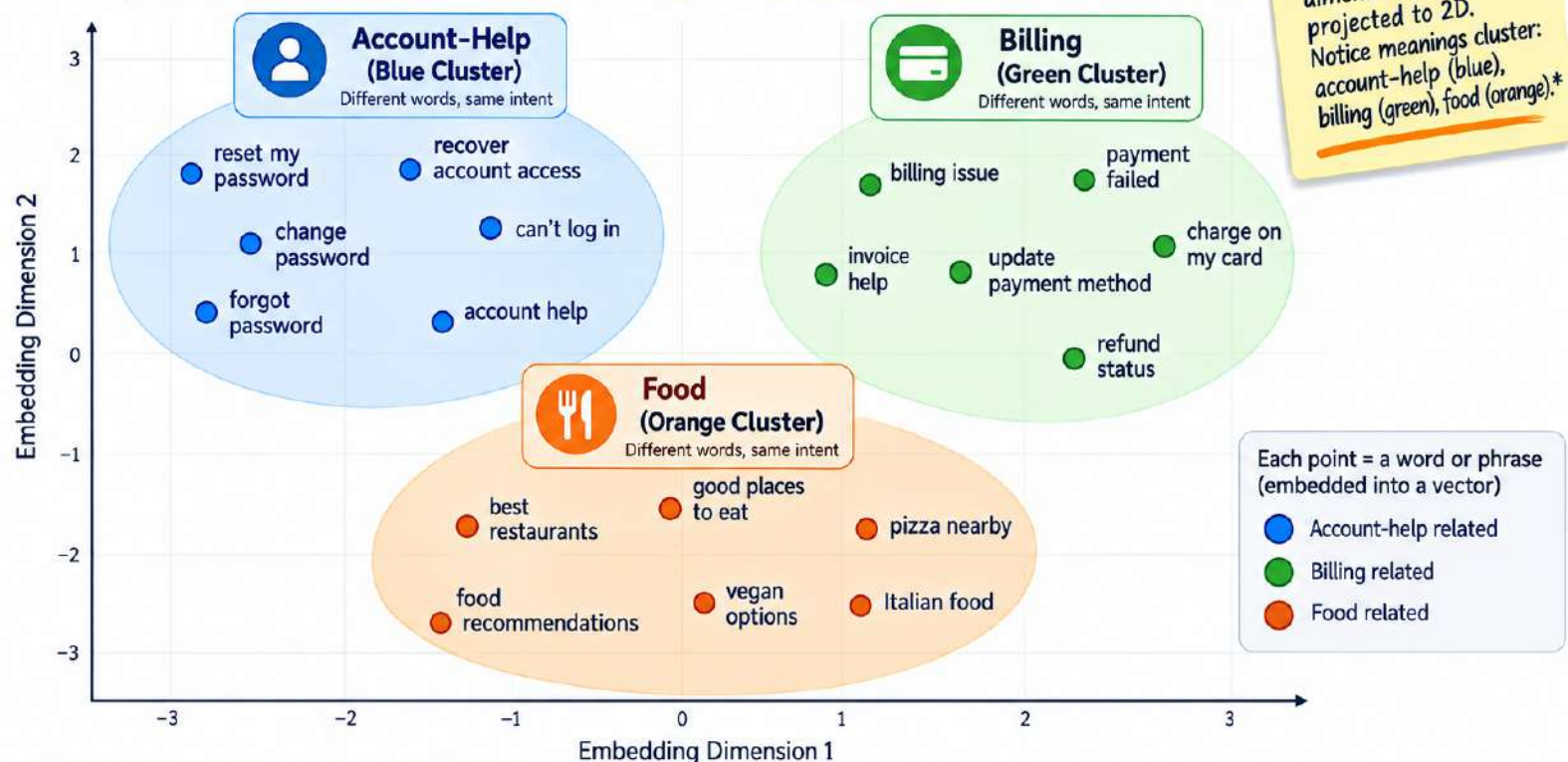
Same Pipeline, Different Input

- Documents → index (done once)
- User query → search (every request)

Same model. Same process. Different input.

Embeddings: Similar Meanings Cluster Together

Different words, same meaning → close in vector space



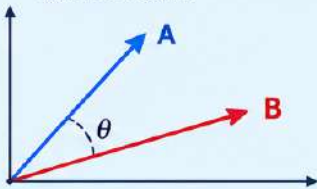
Key Takeaway: Words with similar meaning end up close in vector space, even if they don't share exact words. This is why similarity search can understand intent, not just keywords.

Measuring Similarity Between Vectors

Same vectors, different views — direction, magnitude, or straight-line gap

1 Direction Only

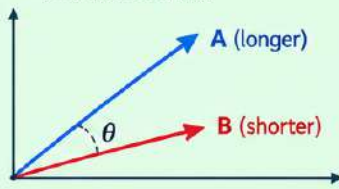
Which way the vector is pointing (ignores length).



If two vectors point in the same direction (small angle θ), they are more similar — even if their lengths are different.

2 Direction + Magnitude

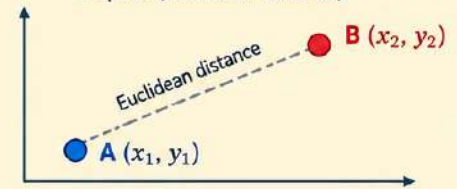
Which way + how long the vector is (considers length).



Here we care about both the direction and the size (magnitude) of the vectors.

3 Straight-Line Gap

How far apart the two points are in space (Euclidean distance).



Measures the straight-line distance between two vectors. Smaller distance means more similar.

4 Pick a Metric

Different metrics capture different notions of similarity.

Metric	What it measures	When to use	Formula	Visual	Notes
Cosine similarity	Angle between vectors (direction only)	✓ Default for text (semantics matter more than length)	$\cos(\theta) = \frac{A \cdot B}{\ A\ \ B\ }$		- Range: $[-1, 1]$ 1 = same direction (more similar)
Dot product	Direction + magnitude	Fast; use if vectors are normalized	$A \cdot B = \sum A_i B_i$		- Larger value = more similar - Fast to compute
Euclidean (L2)	Straight-line gap (distance)	Magnitude-sensitive (useful for non-normalized vectors)	$\ A - B\ _2 = \sqrt{\sum (A_i - B_i)^2}$		- Smaller distance = more similar - Always ≥ 0

5 Important Note

If vectors are normalized ($\|A\| = \|B\| = 1$),

cosine $\approx$ dot product $\approx$ L2 ranking
(they give the same nearest neighbors)

💡 Same data, different views, same neighbors (if normalized!)

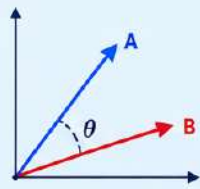
Common Similarity Metrics Between Vectors

Same vectors, different views — direction only, direction + magnitude, or straight-line gap

1

Cosine Similarity

Direction only (angle between vectors)



$$\text{cosine}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

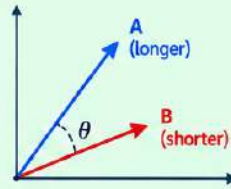
Measures the angle between vectors (ignores length). Values range from -1 to 1. 1 = same direction (more similar).

Even if the lengths are different, vectors pointing in the same direction have high cosine similarity.

2

Dot Product

Direction + magnitude



$$A \cdot B = \sum_i A_i B_i$$

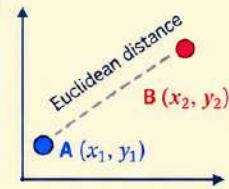
Considers both the direction and the lengths (magnitude) of the vectors. Larger value = more similar (if vectors are normalized).

Fast to compute; commonly used when vectors are normalized.

3

Euclidean Distance (L2)

Straight-line gap (distance between vectors)



$$\text{Euclidean}(A, B) = \sqrt{\sum_i (A_i - B_i)^2}$$

Measures the straight-line distance between two vectors. Smaller distance means more similar.

Takes magnitude into account and is useful when vectors are not normalized.

4

Quick Comparison

Metric	What it measures	Formula	Best used when	Visual	Notes
Cosine similarity	Angle between vectors (direction only)	$\text{cosine}(A, B) = \frac{A \cdot B}{\ A\ \ B\ }$	✓ Default for text (semantic similarity)		- Range: $[-1, 1]$ 1 = same direction (more similar) - Ignores vector length
Dot product	Direction + magnitude	$A \cdot B = \sum_i A_i B_i$	Fast; use if vectors are normalized		- Larger value = more similar - Fast to compute - If normalized, ranking same as cosine & L2
Euclidean (L2)	Straight-line gap (distance)	$\text{Euclidean}(A, B) = \sqrt{\sum_i (A_i - B_i)^2}$	Magnitude-sensitive (useful for non-normalized vectors)		- Smaller distance = more similar - Always ≥ 0 - Sensitive to vector length

5

Key Takeaway

If vectors are normalized ($\|A\| = \|B\| = 1$), **cosine similarity $\approx$ dot product $\approx$ L2 ranking** (they give the same nearest neighbors).

$$\text{cosine}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

$$\text{Euclidean}(A, B) = \sqrt{\sum_i (A_i - B_i)^2}$$



Different formulas, same idea — measure how similar two vectors are!

Finding Similar Vectors: Exact kNN vs Approximate ANN

Same goal — different approach, different speed vs accuracy trade-off

Use exact when you need 100% accuracy.
Use approximate when you need speed at scale!

1 Query Vector

Convert the user query into a vector using the same embedding model.

"How can I reset my password?"

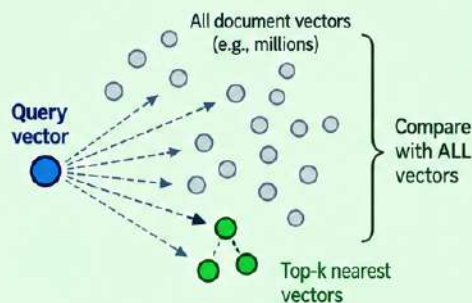
Embedding Model
(same as documents)

0.12 -0.45 0.33 ...

Query vector
(d dimensions, e.g., 768)

2 Exact kNN

Compare the query to **ALL** vectors in the collection.



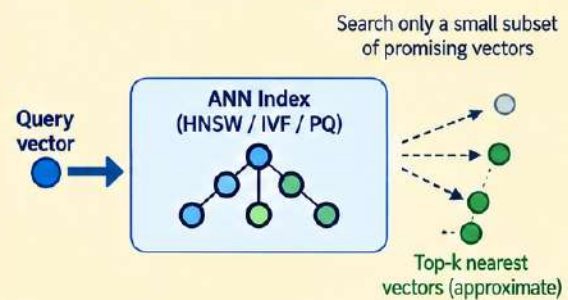
✓ **100% accurate**
(find true nearest neighbors)

✗ **Slow at millions of vectors**
(brute force comparison)

Good for small datasets or when perfect accuracy is required.

3 Approximate ANN

Use an index (e.g., HNSW, IVF, PQ) to search only a small, promising subset of vectors.



✓ **Millisecond at billions of vectors**
(super fast, scalable)

⚠ **~99% recall**
(might miss a few nearest neighbors)

Good for large-scale systems (RAG, search, recommendations, etc.).



Key Takeaway

Both methods use the same query vector and similarity metric (e.g., cosine similarity).
The difference is how many vectors we compare and how we search for the nearest neighbors.

Same vector.
Different search strategy.
Choose what fits your scale and accuracy needs!

HNSW (Hierarchical Navigable Small World)

Find similar vectors by greedy search through multiple layers

Start high,
zoom in,
find the closest!

Top layer
(few nodes,
far apart)

(e.g., layer 2)

Middle layer
(more nodes,
closer together)

(e.g., layer 1)

Bottom layer
(many nodes,
densest)

(e.g., layer 0)

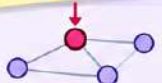
Start here
(enter at top layer)

Query
(vector)

1

Enter at top layer

Start from a few, far-apart nodes
(at the highest layer).



2

Greedy hop toward query

At the current layer, move to the
neighbor that is closer to the query
(greedy search).



3

Drop to denser layer

Go down one layer (more nodes,
closer together).



4

Hop closer again

Greedy move to a closer neighbor
at this layer.



5

Bottom layer: refine to k nearest

At the bottom layer (densest),
explore nearby nodes thoroughly
to find the top-k nearest neighbors.



6

Return top-k

Output the k most similar vectors
from the candidates found.

Top-k results

1	Doc A	(0.92)
2	Doc B	(0.89)
3	Doc C	(0.87)
4	Doc D	(0.83)
5	Doc E	(0.80)



Why it works

- **Small world property:** few hops to reach any node.
- **Hierarchical layers:** fast coarse search at top, fine search at bottom.
- **Greedy search:** simple and efficient.



Benefits

- Very fast (millisecond search at large scale)
- High recall (often ~99%+)
- Works well for high-dimensional vectors
- Widely used in vector databases (e.g., Pinecone, Milvus, Weaviate, etc.)



Key Takeaway

HNSW searches from a sparse top layer to a dense bottom layer, greedily moving closer to the query at each step, and finally returns the top-k nearest neighbors.

Same idea as navigating a city: look at the big map first, then zoom into local streets!

Code Implementation: Similarity Search

From simple (NumPy) → to real ANN (FAISS) → to best of both (Hybrid)

A Pure-NumPy similarity search (understand the core)

We'll use small 3D vectors to keep it simple. In real life, embeddings are 100s–1000s of dimensions.

Documents (embeddings)

Doc 0: "reset password" [0.9, 0.1, 0.2]
Doc 1: "forgot login" [0.8, 0.2, 0.1]
Doc 2: "pizza recipe" [0.1, 0.9, 0.7]

Query
"account locked"
[0.85, 0.15, 0.15]

Normalize vectors
Compute similarity
(cosine = dot for normalized)

Rank and return top results

Python (NumPy)

```
import numpy as np

def normalize(v):
    return v / np.linalg.norm(v, axis=-1, keepdims=True)

docs = normalize(np.array([
    [0.9, 0.1, 0.2], # "reset password"
    [0.8, 0.2, 0.1], # "forgot login"
    [0.1, 0.9, 0.7], # "pizza recipe"
]))

query = normalize(np.array([0.85, 0.15, 0.15])) # "account locked"

scores = docs @ query # cosine == dot for normalized
order = np.argsort(-scores) # highest first
print(order, scores[order].round(3))

# Output: [0 1 2] -> login docs win
# [0 1 2] [0.997 0.980 0.404]
```



Simple, easy to understand. Good for learning and small datasets.

B Real ANN index with FAISS (fast, scalable)

FAISS is a library by Meta for efficient similarity search on large datasets (millions/billions of vectors).

Document embeddings (N x 1536)

[...]
[...]
[...]

Add to index

FAISS Index
(e.g., IndexFlatIP)

Query embedding (1 x 1536)

[...]

Search (top-k)

Top-k similar documents
(scores + ids)

Python (FAISS)

```
import faiss, numpy as np

dim = 1536
index = faiss.IndexFlatIP(dim) # inner-product # (cosine if normalized)

# vectors = your normalized doc embeddings, shape (N, dim)
index.add(vectors)

faiss.normalize_l2(query) # normalize query too
scores, ids = index.search(query, k=5) # top-5 nearest
print(ids)
print(scores)

# Output (example):
# [[12 7 98 21 5]]
# [[0.87 0.86 0.84 0.82 0.81]]
```



Super fast (milliseconds) and works at scale (millions/billions of vectors).

C Hybrid search (keyword + semantic) (best recall)

Use keyword search (e.g., BM25) to catch exact terms + vector search to catch meaning. Then merge and rerank for best results.

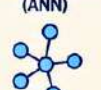
1. Keyword search (BM25)



Find exact term matches
(e.g., "reset password")

Top-20 keyword hits
(doc IDs + scores)

2. Vector search (ANN)



Find semantic matches
(e.g., "account locked")

Top-20 vector hits
(doc IDs + scores)

3. Merge + Rerank



Merge unique candidates
and rerank (e.g., cross-encoder)
for best results.

Final top-5 results
(most relevant)

Python (Hybrid Search)

```
# 1) BM25 keyword candidates
kw_hits = bm25.search(query_text, k=20) # exact terms, doc IDs

# 2) Vector candidates
vec_hits = index.search(embed(query_text), k=20) # semantic matches

# 3) Merge + rerank
candidates = merge_unique(kw_hits, vec_hits)
final = reranker.rank(query_text, candidates)[:5] # cross-encoder
print(final)

# Output (example):
# [
#   {'id': 12, 'score': 0.92, 'text': 'How to reset your password'},
#   {'id': 7, 'score': 0.89, 'text': 'Account recovery steps'},
#   ...
# ]
```



Highest recall and best user experience. Combines exact keywords + semantic meaning.



Key Takeaway

Start simple (NumPy) to understand the core → use FAISS for scale → use Hybrid (keyword + semantic + rerank) for the best real-world results.

Same idea,
different tools,
choose based on your
scale and accuracy needs!

End-to-End Similarity Search Pipeline (RAG)

From data ingestion to real answers — powered by vector search

Same idea, different stages.
Ingest → Store → Search →
Rerank → Answer → Monitor

1 Ingest Pipeline

Turn your raw data into vectors and store them in a vector DB.

Raw Data



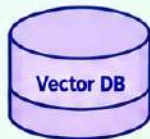
Chunk
Split into smaller chunks (e.g., 500–1,000 tokens)

Embed
Convert each chunk to a vector using an embedding model (e.g., OpenAI, Instructor, SBERT, etc.)

Upsert
Store vectors + metadata in a Vector Database (e.g., Pinecone, Weaviate, Qdrant, FAISS, etc.)

2 Vector Database

Store and index vectors for fast similarity search.



(Pinecone / Weaviate / Qdrant / FAISS / etc.)

HNSW index

- HNSW (Hierarchical Navigable Small World)
- Fast and scalable
- Supports metadata filtering
- Handles millions/billions of vectors

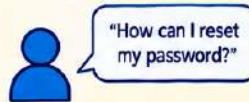
Store

- Vector embeddings
- Original text (chunk)
- Metadata (title, source, tags, etc.)

3 Query (User Request)

Convert the user's question into a vector and search for similar content.

User Query



Embed Query

Convert the query to a vector (using the same embedding model as documents).

ANN Search (Top-k)

Search the HNSW index to find the top-k most similar chunks (e.g., k = 5 or 10).

4 Rerank (Optional)

Improve accuracy by re-ranking the retrieved results.

Top-k Candidates

Chunk 1	(score: 0.92)
Chunk 2	(score: 0.87)
Chunk 3	(score: 0.83)
Chunk 4	(score: 0.80)
Chunk 5	(score: 0.76)

Rerank with Cross-Encoder

Use a stronger model (e.g., BERT cross-encoder) to re-rank based on deeper relevance.

Top Results (re-ranked)

1	Chunk 3	(score: 0.95)
2	Chunk 1	(score: 0.91)
3	Chunk 4	(score: 0.87)

5 Context → LLM (RAG)

Use the most relevant content to generate a grounded answer.

Build Context



- Combine top-k chunks
- Add instructions / system prompt
- Include metadata (optional)

LLM (e.g., GPT-4, Claude, etc.)

Generate a final answer using the retrieved context (RAG – Retrieval Augmented Generation).

Grounded Answer

"To reset your password, go to Settings → Account → Reset Password. You'll receive a link on your registered email."

(with source links)

Source 1 Source 2 Source 3

6 Monitor & Improve

Keep the system accurate, fast, and reliable.



Recall / Relevance
Are the right results being retrieved?



Latency
Is search fast enough (e.g., < 100 ms)?



User Feedback
Track clicks, ratings, helpful/not helpful.



Data Drift
Monitor for outdated content or embedding degradation.



Logging & Analytics
Track queries, results, and performance metrics.



Periodic Re-embedding
Re-embed content when models or data change.



Key Takeaway

Chunk → Embed → Upsert →
ANN Search → (Rerank) →
Context → LLM → Monitor

Same idea, different stages.
Together they deliver accurate,
relevant, and scalable answers!

Which Search Approach Should You Use?

Match your use case to the right search strategy

Different needs require different search methods. Often, the best results come from combining them!

1 Exact term / ID / SKU / code

Users search for an exact word, product code, ID, or known term.

Q iPhone 15

Examples:

- Product SKU: "SKU12345"
- Order ID: "ORD-9876"
- Error code: "ERR_404"
- Brand / model: "iPhone 15"

Keyword / BM25



- ✓ Precise matching
- ✓ Cheap and fast
- ✓ Explainable results

Best for exact terms, IDs, codes, names.

2 Meaning / paraphrase / questions

Users may ask in natural language or use different words.

Q How can I reset my password?

Examples:

- "Forgot my login"
- "Can't access my account"
- "Refund process"
- "Best laptop for students"

Semantic (embeddings)



- ✓ Understands meaning
- ✓ Handles paraphrases
- ✓ Works with natural language questions

Best for conceptual / semantic search.

3 Numeric / date / category filter

Users want to filter or search by structured data.

Q laptops under 50000

Examples:

- Price < 50000
- Launch date Gfter 2024-01-01
- Category = "Laptops"
- Rating >= 4

SQL / metadata filter



- ✓ Fast and accurate
- ✓ Handles numeric, date, category filters
- ✓ Uses structured data

Best for filtering structured fields.

4 Need both precision & recall

Users expect relevant results even with different wording, plus filters.

Q best laptop under 50000 for coding

Examples:

- Natural language + filters
- "Affordable iPhone with good camera"
- "Refund policy for order ORD-9876"
- "Show me red shoes size 9"

Hybrid + rerank



- ✓ Best of both worlds
- ✓ High precision + high recall
- ✓ Rerank with stronger model (e.g., cross-encoder)

Production default for most real-world systems.



Key Takeaway

Think about what kind of match your users need, then choose the right approach.

For most real-world applications, a hybrid approach (keyword + semantic + filters + rerank) gives the best results.

Right tool for the right need. Combine methods to deliver relevant, accurate, and helpful results!

Vector Search & RAG – One Page Visual Revision

From Text to Answer – Key concepts, flow, formulas and tips

Learn Once
Revise Fast
Ace Your Interviews!

1. What is a Vector?

- A list of numbers representing meaning of text, image, etc.
- Example (3 dims): [0.1, 0.5, -0.3]
- Real models: 768, 1536, 3072 dims

2. Similarity Metrics

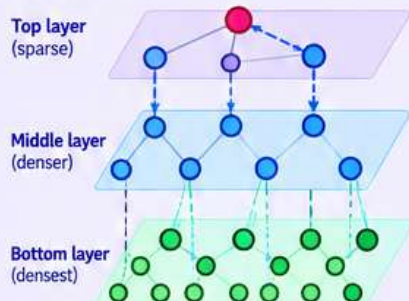
- Cosine similarity – most common (for normalized vectors)
- Dot product – same as cosine if vectors are normalized
- L2 distance – Euclidean distance (smaller = more similar)

3. Exact kNN vs ANN

Exact kNN	ANN
Compare with all N	Use smart index
Exact results (100%)	Approximate (≈99%+)
Slow (O(N))	Very fast
Not scalable	Scales to billions

4. HNSW Intuition

Multi-level graph. Start from top layer, make large jumps, go to lower layers, refine neighbors.



5. Scaling Techniques

- Quantization (e.g., 8-bit)
- Product Quantization (PQ) / IVF
- Dimension reduction
- Disk-based indexes
- Sharding (split data)
- Replication (high availability)
- Filtering (metadata)
- Caching (frequent queries)

6. Evaluation Metrics

- **Recall@K** – relevant result in top-K?
- **MRR** – first relevant result rank
- **nDCG** – ranking quality (graded relevance)



TEXT

(Query or Document)

"How to reset my password?"



EMBEDDING

Convert text to dense vector using an embedding model (e.g., OpenAI, BERT, SBERT)



VECTOR

[0.12, -0.34, 0.56, ...]
(e.g., 1536 dimensions)

SIMILARITY METRIC

Measure how similar two vectors are



COSINE

(sim(v1, v2))

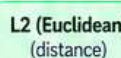
$$\frac{v1 \cdot v2}{||v1|| \cdot ||v2||}$$



DOT PRODUCT

(v1 · v2)

$$\sum v1_i \cdot v2_i$$



L2 (Euclidean)

(distance)

$$\sqrt{\sum_i (v1_i - v2_i)^2}$$



SEARCH

Find similar vectors in a large dataset

Exact kNN

Compare with ALL vectors (brute force)

- ✓ Exact results (maximum recall)
- ✗ Slow and expensive at scale
- ✗ Not suitable for millions/billions

ANN (Approximate NN)

Use a smart index to search in promising regions

- ✓ Very fast (millisecond latency)
- ✓ Scales to millions/billions
- ✗ Small recall trade-off



HNSW

Hierarchical Navigable Small World (multi-level graph index)



Top-K

Retrieve top K most similar chunks (e.g., K = 5, 10, 20)



Hybrid Search / Rerank

Combine keyword (BM25) + semantic search and rerank using a stronger model (e.g., cross-encoder)



Context

Select top relevant chunks to provide to the LLM (RAG)



LLM

Generate answer using the provided context (e.g., GPT-4, Claude, Llama)



Answer

Relevant, grounded, and accurate response to the user

7. Ingest Pipeline (Documents)



Load documents (PDFs, web pages, etc.)



Chunk (e.g., 500–1,000 tokens)



Generate embeddings



Store in vector DB (e.g., Pinecone, Weaviate, Qdrant, FAISS) with HNSW index



Add metadata (IDs, filters, etc.)

8. Query Pipeline (User)



User query



Embed query (same model)



Search in vector index (ANN)



Get top-K results



Rerank (optional)



Pass context to LLM (RAG)



Generate answer

9. When to Use What?

User need	Best option
Exact term / ID / SKU	Keyword / BM25
Meaning / paraphrase	Semantic (embeddings)
Numeric / date filter	SQL / metadata filter
Need both precision & recall	Hybrid + rerank (Production default)

10. Troubleshooting Retrieval

Relevant doc missing?	Check chunking, embedding, Top-K, ANN params, try hybrid
Doc retrieved but low rank?	Add/tune reranker
Wrong customer's data?	Fix metadata authorization
Exact ID query?	Add keyword/DB retrieval

11. Storage Calculation

Raw vector storage (no index, no metadata):

Number of vectors × Dimensions × bytes/value

Example:

1B vectors × 1536 dims × 4 bytes
≈ **6.144 TB raw vector values**

Real system: index + metadata + replication overhead (typically 2–5× or more).

12. Key Takeaway



- Same embedding model for documents & queries
- Tune the right knobs (chunking, Top-K, ANN, reranker)
- Evaluate with Recall@K, MRR, nDCG
- Use hybrid search for best real-world results
- Plan for scale (compression, sharding, caching)
- Good retrieval = better, more grounded LLM answers

Vector Storage & Scaling for Real-World Systems

How much space do vectors take? What's the real cost? How do we scale?

Vectors are small, but at scale they are HUGE! Use the right techniques to store and search efficiently.

1 Raw Vector Storage Intuition

Total storage for vector values (no index, no metadata)

Number of vectors × Dimensions × bytes/value

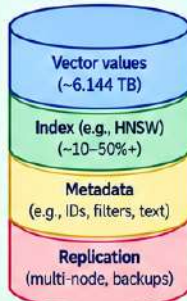
Example

1B vectors × 1536 dims × 4 bytes
≈ **6.144 TB raw vector values**

Vectors	Dimensions	Bytes/value	Total Size
1,000,000,000	1536	4	≈ 6.144 TB

2 Why Extra Storage?

Real systems have additional overhead



- Index structures (e.g., HNSW graph)
- Metadata (IDs, filters, payload)
- Replication (for high availability)
- Snapshots / backups
- Additional system overhead

So actual storage can be 2-5x (or more) than raw vector values.

3 Example with Overhead

If raw size is ~6.144 TB, then with index, metadata, and replication:

Actual storage
≈ **12 TB - 30 TB+**

(depending on index type, replication factor, and metadata)

Example breakdown (illustrative):

Component	Size (approx)
Vector values	6.144 TB
Index (HNSW, ~50%)	3-6 TB
Metadata (IDs, filters)	0.5-2 TB
Replication (2-3x)	10-20+ TB total

4 Key Takeaway



Vectors are lightweight individually, but at billion scale, storage and index overhead become significant.

Plan for scaling from day one!

"A system that works for millions may not work for billions. Design with scale in mind."



Scaling Techniques

Use these techniques to reduce storage, improve search speed, and handle large-scale vector data.

1 Quantization

Use lower precision (e.g., 8-bit instead of 32-bit).



- Reduces storage by 4x (or more)
- Small accuracy loss
- Common in production

2 PQ / IVF

Product Quantization (PQ) and Inverted File (IVF) indexes.



- Compresses vectors
- Search only in relevant clusters (IVF)
- Much smaller index size
- Widely used (e.g., FAISS)

3 Dimension Reduction

Reduce vector dimensions (e.g., 1536 → 256).



- Less storage
- Faster search
- Techniques: PCA, autoencoders, model-specific embeddings

4 Disk-based Indexes

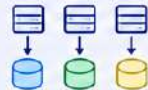
Store index on SSD/disk instead of RAM.



- Handle billions of vectors
- Lower RAM cost
- Slightly higher latency (but still fast)
- Examples: DiskANN, Vamana

5 Sharding

Split data across multiple machines/nodes.



- Horizontal scaling
- Handle massive datasets
- Distribute read/write load
- Common in production vector DBs

6 Replication

Keep multiple copies for high availability.



- Fault tolerance
- High availability
- Better read performance
- Increases storage cost (typically 2-3x)

7 Filtering

Use metadata filters to search in a subset.



- Filter by user_id, category, timestamp, etc.
- Reduces search space
- Faster and more relevant results

8 Caching

Cache frequent queries and results.



- Lower latency
- Reduces compute cost
- Good for popular queries
- Use in-memory cache (Redis, etc.)



Final Takeaway

At scale, vector storage is not just about the raw numbers — you need the right combination of compression, indexing, sharding, replication, and caching to build a cost-efficient and high-performance system.

Right technique.
Right scale.
Real-world success!

Retrieval Bad? Troubleshoot Step by Step

Find the root cause. Check the right things. Fix it with the right solution.

Most retrieval issues have clear causes. Check systematically, not randomly!

Retrieval bad

Results are not relevant or missing expected information.

Common symptoms:

- Irrelevant results
- Relevant document missing
- Wrong data / user data
- Correct document but low rank
- Exact query not working

1 Relevant document missing?

The right document is not retrieved in top-K results.

Check and try these:

- **Chunking check** (size, overlap, structure)
- **Embedding check** (model, dimension, domain fit)
- **Top-K check** (increase K, see if it appears)
- **ANN parameters** (e.g., HNSW efSearch, IVF params)
- **Try hybrid** (keyword + semantic search)

Goal

- Ensure the document can be found by the retrieval system.
- Expand recall by tuning index and using hybrid search.

2 Document retrieved but low rank?

The relevant document is in results but ranked too low.

Solution:

- **Add or tune reranker**
 - Use a stronger model (e.g., cross-encoder)
 - Tune ranking features (title, content, metadata)
 - Combine multiple signals (semantic + keyword + metadata)

Goal

- Improve ranking quality (MRR, nDCG).
- Bring relevant documents to top positions.

3 Wrong customer's data?

Retrieved document belongs to a different user/tenant.

Solution:

- **Fix metadata authorization**
 - Add user/tenant filters (e.g., user_id, org_id)
 - Enforce access control in retrieval layer
 - Validate metadata during indexing and query time

Goal

- Ensure data privacy and isolation.
- Return only authorized documents.

4 Exact ID query?

User is searching for an exact term, ID, SKU, or code.

Solution:

- **Add keyword/DB retrieval**
 - Use BM25 / keyword search for exact matches
 - Query structured database (SQL) if needed
 - Combine with semantic search (hybrid)

Goal

- Get precise and deterministic results.
- Handle exact queries efficiently.

Key Takeaway

Retrieval issues usually fall into a few clear categories. Check systematically: missing document → low rank → wrong data → exact query.

Use the right tool (chunking, embeddings, Top-K, ANN, hybrid, reranker, or metadata filters) to fix it.

Better retrieval = better answers = happier users!

Retrieval Troubleshooting Guide

Find the root cause. Try the right fix. Get better results.

Retrieval issues are common. Follow a structured approach to debug and fix them!



Not getting the right documents or results.



Relevant document missing?

The document you expect is not retrieved at all.

- 1 Chunking check
- 2 Embedding check
- 3 Top-K check
- 4 ANN parameters
- 5 Try hybrid (keyword + semantic)

What to do:

- Use smaller / better chunks
- Check if the content is actually in your data
- Try a stronger embedding model
- Increase Top-K
- Tune ANN parameters (e.g., HNSW efSearch, IVF nprobe)
- Add keyword search (BM25) + semantic search

Example:

"Company leave policy document exists but is not retrieved."



Document retrieved but low rank?

The relevant document is in results but ranked low.

- 1 Add / tune reranker
 - Use a stronger reranker model (e.g., cross-encoder)
 - Tune rerank top-N (e.g., rerank top 20)
 - Use hybrid scores (BM25 + dense)
 - Adjust similarity metric / weighting

What to do:

- Use a cross-encoder or LLM-based reranker
- Tune reranker parameters
- Consider hybrid search for better candidates

Example:

"The correct document is in top 20 but at rank 15."



Wrong customer's data?

Retrieved document belongs to a different user / organization.

- 1 Fix metadata authorization
 - Add proper filters (tenant_id, user_id, org_id)
 - Enforce access control in retrieval layer
 - Verify metadata in chunks and queries
 - Test with multiple users/tenants

What to do:

- Use metadata filtering (e.g., where tenant_id = X)
- Validate authorization logic
- Add strict filters before semantic search
- Test with different user accounts

Example:

"User A is seeing User B's private data."



Exact ID query?

User is searching for a specific ID / SKU / code.

- 1 Add keyword / DB retrieval
 - Use keyword search (BM25 / inverted index)
 - Or direct database lookup (SQL)
 - Combine with semantic search (hybrid)
 - Detect ID-like queries and route to keyword/DB

What to do:

- Identify patterns (ID, SKU, code, order number)
- Use exact match search (keyword or SQL)
- Optionally merge with semantic results
- Always prefer precision for ID queries

Example:

"Order ID ORD-9876 should return the exact order details."



Key Takeaways

- ✓ Identify the issue category (missing, low rank, wrong data, or exact ID).
- ✓ Apply the right fix based on the root cause.
- ✓ Use a combination of techniques: better chunking, embeddings, Top-K, ANN tuning, hybrid search, reranking, and metadata filters.
- ✓ Test with real queries and monitor metrics (Recall@K, MRR, nDCG) to validate improvements.

Debug step by step. Small changes can lead to big improvements!

Evaluate & Optimize Your Search / RAG System

Measure what matters. Tune the right knobs. Get better results.

Measure → Understand
Tune → Improve
Better Search =
Happier Users!



Key Evaluation Metrics

How to measure the quality of your search / RAG results.

1

Recall@K

Relevant result top-K mein mila?

$$\text{Recall@K} = \frac{\# \text{ relevant results in top-K}}{\# \text{ total relevant results}}$$

- Tells you if you are able to find relevant items.
- Higher is better (0 to 1).
- Important for ensuring coverage.

Example

Total relevant = 5, relevant in top-5 = 4
 $\text{Recall@5} = \frac{4}{5} = 0.8$

Focus:

Don't miss
relevant results!

2

MRR (Mean Reciprocal Rank)

First relevant result kitni early aaya?

$$\text{MRR} = \frac{1}{N} \sum_{i=1}^N \frac{1}{\text{rank}_i}$$

- Measures how early the first relevant result appears.
- Higher is better (0 to 1).
- Good for user satisfaction (early good results).

Example

First relevant at rank 3
 $\text{RR} = 1/3 = 0.33$
(Average over all queries = MRR)

Focus:

Get relevant results
earlier!

3

nDCG (Normalized Discounted Cumulative Gain)

Ranking quality with graded relevance.

$$\text{DCG@K} = \sum_{i=1}^K \frac{(2^{\text{rel}_i} - 1)}{\log_2(i + 1)}$$

$$\text{nDCG@K} = \frac{\text{DCG@K}}{\text{IDCG@K}}$$

- Considers the position and the relevance level (graded relevance).
- Higher is better (0 to 1).
- Good for evaluating overall ranking quality.

Example

Top-3 relevance scores: [3, 2, 1]
 $\text{DCG@3} = (2^3 - 1)/\log_2(2) + (2^2 - 1)/\log_2(3) + (2^1 - 1)/\log_2(4)$
 $\text{nDCG@3} = \text{DCG@3} / \text{IDCG@3}$

Focus:

More relevant items
at higher ranks!



Tuning Knobs

Tune these parameters to improve recall, relevance, and overall performance.

1

Chunking

How you split documents into smaller pieces.



- Chunk size (e.g., 500–1,000 tokens)
- Overlap (e.g., 50–200 tokens)
- Structure-aware chunking (headings, sections)

Better chunks = more relevant and complete context.

2

Embedding model

Model to convert text into vectors.



- Model choice (e.g., OpenAI, SBERT, BGE, E5, etc.)
- Embedding dimension (e.g., 768, 1536)
- Domain-specific vs general model
- Normalization (for cosine)

Better embeddings = better semantic understanding.

3

Top-K

How many results to retrieve from the index.



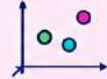
- K value (e.g., 5, 10, 20)
- Higher K → better recall (but more cost)
- Lower K → faster (but may miss results)

Tune K based on your use case and latency/cost requirements.

4

Similarity metric

How to measure distance between vectors.



- Cosine similarity (most common)
- Dot product (for normalized vectors)
- Euclidean / L2 distance
- Choice depends on model and index.

Use cosine for most embedding models.

5

ANN parameters

Settings for the vector index (e.g., HNSW, IVF, PQ).



- HNSW: efSearch, M
- IVF: nlist, nprobe
- PQ: number of subquantizers, bits
- Trade-off: higher values → better recall, higher latency and memory.

Tune for your scale and latency/recall target.

6

Hybrid weighting

Combine keyword (BM25) and semantic search.



- Weighting formula (e.g., $\alpha \cdot \text{BM25} + (1 - \alpha) \cdot \text{semantic}$)
- Tune weight (α) based on dataset
- Can also use learned fusion (e.g., RRF, LTR)

Hybrid search = best of exact match + meaning.

7

Reranker

Use a stronger model to rerank top-K results.



- Model choice (e.g., BERT cross-encoder)
- Rerank top-N (e.g., 50 → 5)
- Domain-specific reranker
- Trade-off: higher accuracy, more latency and cost.

Reranker improves precision and overall ranking quality.



Key Takeaway

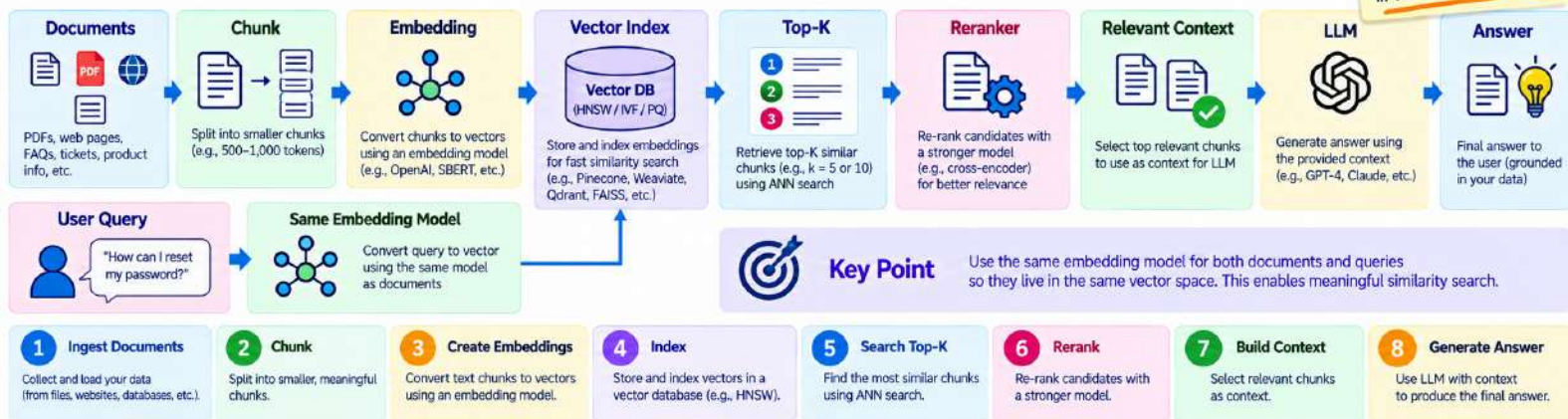
Use the right metrics (Recall@K, MRR, nDCG) to measure performance, and tune these knobs (chunking, embedding model, Top-K, similarity metric, ANN parameters, hybrid weighting, reranker) to get the best results for your use case.

Measure → Tune → Iterate
That's how you build a
high-performing search / RAG system!

End-to-End Flow: From Documents to Answer (RAG)

Ingest → Index → Search → Rerank → Context → LLM → Answer

Same embedding model for documents and query
→ keeps everything in the same vector space!



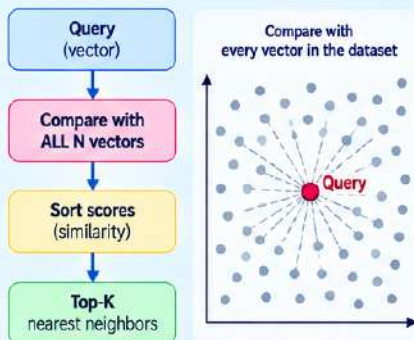
Exact kNN vs ANN (Approximate Nearest Neighbor)

Same goal: find the most similar vectors. Different approach, different trade-offs.

Exact = accurate
ANN = super fast
Use the right tool
for your scale!

1 Exact kNN

Compare with ALL vectors (brute force)



Advantage

- Exact results (maximum recall)
- Simple to understand

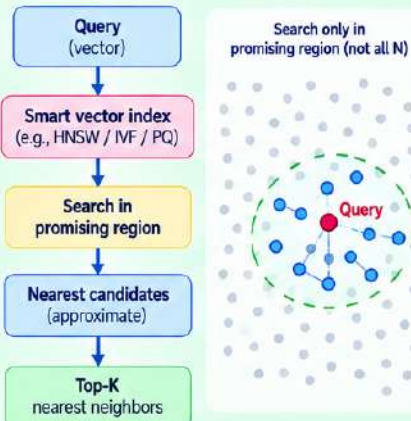


Problem

- Expensive as N (number of vectors) keeps increasing
- Slow at millions/billions of vectors

2 ANN (Approximate Nearest Neighbor)

Use a smart index to search in promising regions



Advantage

- Very fast (millisecond latency)
- Scales to millions/billions of vectors
- Small recall trade-off (~99%+)



Trade-off

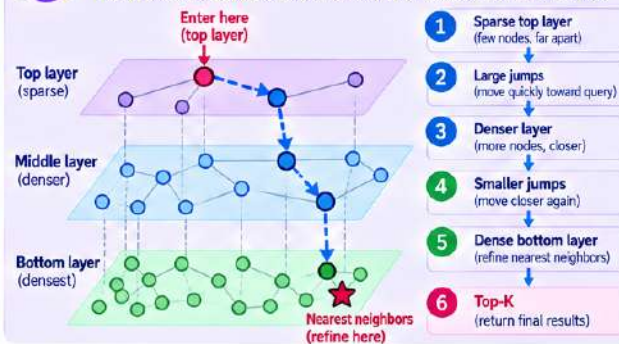
- Approximate results (may miss some neighbors)
- Requires building and tuning an index

3 Quick Comparison

Aspect	Exact kNN	ANN
Search	Compare with all N vectors	Use smart index (e.g., HNSW)
Accuracy / Recall	Exact (100%)	Approximate (~99%+)
Speed (Latency)	Slow (increases with N)	Very fast (milliseconds)
Scalability	Not suitable for millions+ (expensive)	Designed for millions/billions
Best for	Small datasets, evaluation	Real-world systems
Typical use case	Baseline, ground truth	Production search, RAG, recommendations

4 HNSW Intuition

HNSW (Hierarchical Navigable Small World) ko multi-level road map samjho.



Key Trade-off

Exact → accuracy ↑, scale/latency ↓

ANN → speed ↑↑↑, scale ↑↑↑, small recall trade-off



Interview Trigger

"Millions/billions of vectors + millisecond latency"

👉 Use ANN (e.g., HNSW, IVF, PQ)



Key Takeaway

Exact kNN is great for small datasets and evaluation. For real-world large-scale systems (search, RAG, recommendations), ANN with indexes like HNSW is the practical choice.

End-to-End Flow: From User Query to RAG / Recommendation

Find the right information (or items) using vector search and deliver the best results

1 User Query

User enters a question or search request.

"How can I reset my password?"



Can be a:

- Question
- Search term
- Natural language
- Product request

2 Embedding

Convert the query into a vector using an embedding model (same model as docs).

Embedding Model
(e.g., OpenAI, SBERT, BGE, etc.)

[0.12 -0.45 0.33 ...]

Query embedding
(e.g., 1536 dimensions)

3 Query Vector

The query is now a numeric vector that captures its meaning.

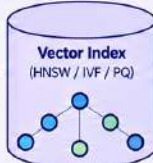
[0.12 -0.45 0.33 ...]



Query vector
(in semantic space)

4 Vector Index

Search the vector index (e.g., HNSW, IVF, PQ) for similar vectors.



Stores document/item embeddings (millions/billions) with an ANN index.

5 Nearest Neighbors

Find the closest vectors in the index using ANN search.

Top-k candidates
(from ANN search)

Doc 1 (score: 0.92)
Doc 2 (score: 0.89)
Doc 3 (score: 0.87)
Doc 4 (score: 0.83)
Doc 5 (score: 0.80)

(k = 5, 10, ...)

6 Top-K

Take the top-k most similar candidates from the ANN results.

Top-K results

1	Doc 1	(0.92)
2	Doc 2	(0.89)
3	Doc 3	(0.87)
4	Doc 4	(0.83)
5	Doc 5	(0.80)

These are the most relevant candidates based on vector similarity.

7 Reranker (optional)

Use a stronger model (e.g., cross-encoder) to rerank the top-k results for better accuracy.

Reranker Model
(e.g., BERT cross-encoder)

Reranked results

1	Doc 3	(0.95)
2	Doc 1	(0.91)
3	Doc 4	(0.87)
4	Doc 2	(0.85)
5	Doc 5	(0.82)

Improves relevance by understanding full context.

Same idea works for search, RAG (chat with your data), or recommendations!

8 RAG / Recommendation

Use the top results as context for an LLM (RAG) or show them as recommendations.

RAG (for Q&A)

Context → LLM → Answer
"To reset your password, go to Settings → Account ..."

Recommendation

Top product recommendations

- Laptop A
- Laptop B
- Laptop C



Key Takeaway

User Query → Embedding → Query Vector → Vector Index → Nearest Neighbors → Top-K → Reranker → RAG / Recommendation
The same pipeline powers semantic search, RAG (chat with your data), and recommendation systems.

Different steps, same goal:
Find the most relevant and useful information!